

RAG Application

Final Report

Generated: August 31, 2026

Technology Stack

Framework: LangChain

LLM: Ollama / llama3.2:latest

Embeddings: Ollama / nomic-embed-text

Vector Database: FAISS

Interface: CLI

1. OVERVIEW

Problem Statement

This project implements a Retrieval-Augmented Generation (RAG) application for answering questions about five seminal research papers in machine learning and natural language processing. The system combines document retrieval with large language model generation to provide accurate, source-attributed answers.

Assignment Objective

The objective is to build a complete RAG pipeline that loads PDF documents, extracts and processes text, generates embeddings, performs semantic search, and generates answers using an open-source language model. The system must maintain conversational context over the last four interactions and be evaluated using predefined questions.

RAG Approach

The application uses LangChain as the RAG framework, Ollama for local LLM and embedding model inference, and FAISS for vector storage. Documents are chunked using RecursiveCharacterTextSplitter and embedded using nomic-embed-text. The system retrieves relevant chunks via semantic similarity and generates answers using llama3.2:latest

Source Documents

Five research papers are used as the knowledge base:

1706.03762.pdf	Attention Is All You Need (Transformer)
1810.04805.pdf	BERT: Pre-training of Deep Bidirectional Transformers
2005.14165.pdf	Denoising Diffusion Probabilistic Models
1907.11692.pdf	LoRA: Low-Rank Adaptation of Large Language Models
1910.10683.pdf	ELECTRA: Pre-training Text Encoders as Discriminators

2. TECHNICAL IMPLEMENTATION

PDF Ingestion

PDFs are loaded using LangChain's PyMuPDFLoader from the data/pdfs/ directory. The loader preserves metadata including source filename, page number, and document ID. All five required PDFs are loaded and validated before processing.

Text Extraction

PyMuPDF extracts text from each page while preserving document structure. Page-level metadata is maintained to enable source attribution in final answers.

Preprocessing

Lightweight text preprocessing normalizes excessive whitespace and removes unnecessary line breaks while preserving technical terminology and meaningful content.

Chunking Strategy

Documents are chunked using RecursiveCharacterTextSplitter with CHUNK_SIZE=800 and CHUNK_OVERLAP=120. Each chunk preserves source metadata and is assigned a unique chunk_id for tracking.

Embedding Generation

Embeddings are generated using Ollama's nomic-embed-text model via langchain_ollama. The model runs locally at http://localhost:11434. The same embedding model is used for both indexing and querying to ensure consistency.

Vector Database

FAISS (Facebook AI Similarity Search) is used for efficient semantic similarity search. The vector store is persisted to data/vectorstore/ and loaded on subsequent runs to avoid regenerating embeddings.

Semantic Retrieval

LangChain's retriever interface provides semantic similarity search with TOP_K=5. Retrieved documents preserve source metadata for attribution.

Prompt Design

A ChatPromptTemplate defines the system prompt with rules for answering primarily from retrieved context, not inventing facts, and clearly stating when information is unavailable. The prompt includes chat history, retrieved context, and the user question.

LLM Generation

Answers are generated using Ollama's llama3.2:latest model via ChatOllama. The model runs locally at http://localhost:11434 with a temperature of 0.7 for balanced creativity.

RAG Pipeline

The RAG chain uses LangChain's LCEL (Runnable interface) to compose retrieval, prompt formatting, and generation into a single pipeline. The chain retrieves relevant documents, formats them into context, and generates answers using the LLM.

Conversational Memory

A `collections.deque` with `maxlen=4` maintains conversation history. Each interaction (question + answer) is stored, and older interactions are automatically removed when the limit is exceeded. Memory is provided to the prompt for contextual follow-up questions.

Source Attribution

Every RAG response displays retrieved source documents and page numbers. The `DocumentFormatter` formats sources for display, using actual metadata from retrieved LangChain Documents.

3. EVALUATION

Evaluation Questions

Ten predefined questions evaluate the system across different aspects of the source documents. Questions include conceptual, factual, comparison, and multi-document queries. Some questions test contextual recall by depending on previous conversation history.

Evaluation Framework

A custom evaluation framework is used due to the local execution environment. The framework evaluates four metrics on a 1-5 scale: Relevance, Accuracy, Contextual Awareness, and Response Quality.

Evaluation Methodology

Metric	Description	Scale
Relevance	How well the answer addresses the question	1-5
Accuracy	Alignment with expected answer and source documents	1-5
Contextual Awareness	Use of conversation history for follow-up questions	1-5
Response Quality	Clarity, structure, and informativeness	1-5

Overall Score

The overall score is calculated as the average of the four metric scores: $(\text{Relevance} + \text{Accuracy} + \text{Contextual Awareness} + \text{Response Quality}) / 4$

4. RESULTS & DISCUSSION

Evaluation Results

Metric	Average Score
Relevance	3.30 / 5
Accuracy	3.40 / 5
Contextual Awareness	3.30 / 5
Response Quality	3.40 / 5
Overall	3.35 / 5

Strengths

- Complete end-to-end RAG pipeline with all required components
- Local execution using Ollama for privacy and cost efficiency
- Efficient FAISS vector storage with persistence
- Conversational memory for contextual follow-up questions
- Source attribution for transparency and verifiability
- Modular architecture with clear separation of concerns

Limitations

- Custom evaluation metrics are heuristic-based rather than using RAGAS or TruLens
- No advanced question contextualization beyond conversation history
- Limited to the five provided PDF documents
- Performance depends on Ollama model availability and hardware
- CLI interface may be less user-friendly than web UI

Challenges

- Ensuring Ollama models are properly installed and running
- Balancing chunk size and overlap for optimal retrieval
- Designing evaluation questions that test different aspects
- Implementing memory that correctly handles the sliding window

5. CONCLUSION

Summary

This project successfully implemented a complete RAG application for answering questions about five machine learning research papers. The system integrates PDF ingestion, text processing, embedding generation, semantic retrieval, and LLM generation using LangChain, Ollama, and FAISS.

Performance

The RAG pipeline performs effectively in retrieving relevant document chunks and generating contextually appropriate answers. The conversational memory enables follow-up questions that reference previous context. Source attribution provides transparency for answers.

Learnings

- LangChain provides a powerful framework for building RAG applications
- Ollama enables local LLM inference without cloud dependencies
- FAISS offers efficient vector similarity search for large document collections
- Conversation memory significantly improves follow-up question handling
- Proper chunking strategy is critical for retrieval quality

Future Improvements

- Integrate RAGAS or TruLens for more sophisticated evaluation
- Implement advanced question contextualization
- Add support for more document types (web pages, text files)
- Develop a web interface using a lightweight framework
- Implement hybrid retrieval combining semantic and keyword search